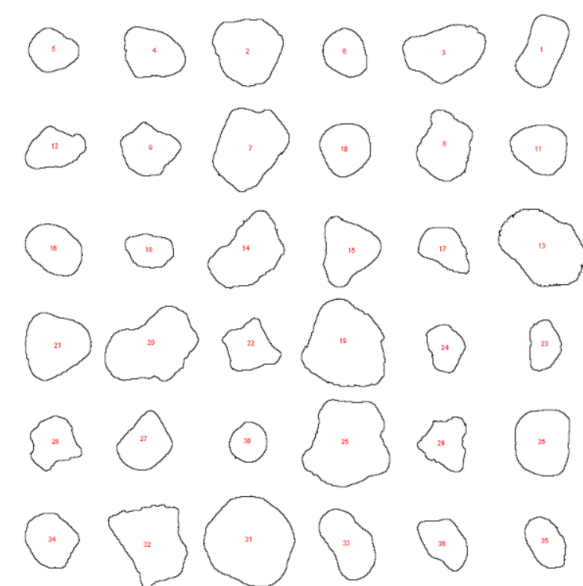
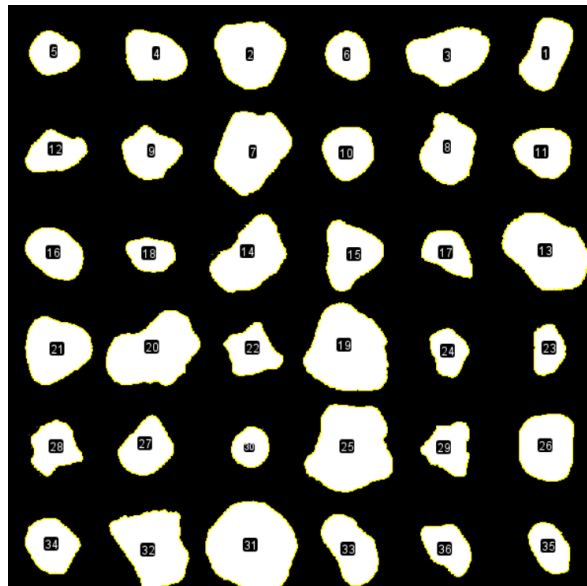
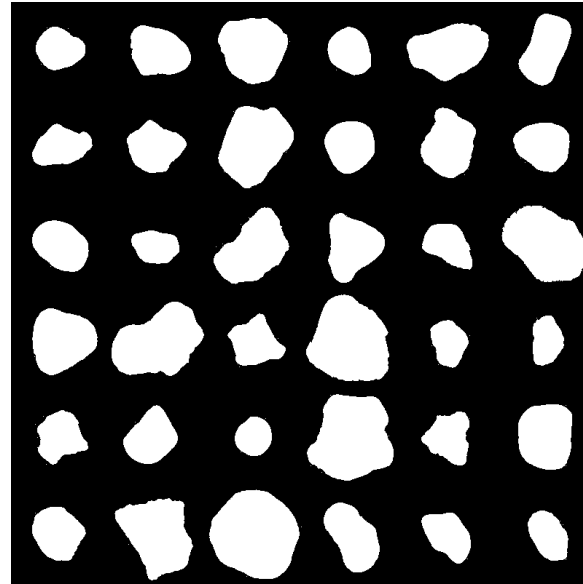




Feature Extraction from Images (ImageJ)

Presented by **Soriano, Edneil E. Jr.**

S.N. 202002420

Objectives

- To identify the objects in an image and isolate them from a background
- To determine various properties of these objects such as the centroids, shape descriptions, area, standard deviation, etc...

ImageJ

ImageJ is a free image processing software written in Java since it mainly concerns itself with analyzing objects. It was developed by the National Institute of Health US and the Laboratory for Optical and Computational Instrumentation (LOCI, University of Wisconsin). Due to this one can easily infer that the software was designed mainly for medical purposes – however, it is so useful and versatile that any field that concerns itself with image processing will find it useful in some way.

The software is opensource and its code readily available in the website. One can download it using the link displayed on the right.

In this activity, we utilize ImageJ for blob detection – that is to separate objects in an image then catalogue each one and their properties.

[home](#) | [docs](#) | [download](#) | [plugins](#) | [resources](#) | [list](#) | [links](#)

Download

Platform Independent

To install ImageJ on a computer with Java pre-installed, or to upgrade to the latest full distribution (including macros, plugins and LUTs), download the [ZIP archive](#) (6MB) and extract the ImageJ directory. Use the *Help>Update ImageJ* command to upgrade to newer versions.

Mac OS X

Download [ImageJ bundled with Java 8](#) (may need to work around [Path Randomization](#)). [Instructions](#). With M1 (ARM) Macs, download [ImageJ bundled with Zulu OpenJDK 13.0.6](#).

Linux

Download [ImageJ bundled with Java 8](#). [Instructions](#).

Windows

Download [ImageJ bundled with 64-bit Java 8](#). [Instructions](#).

Documentation

Tiago Ferreira's comprehensive [ImageJ User Guide](#) is available as an 8MB PDF document and as a [ZIP archive](#). The online [JavaDoc API documentation](#) is also available as a [ZIP archive](#).

Source Code

The ImageJ [Java source](#) consists of 132,000 lines of code in 348 files. It is available [online](#) and as [zip archives](#).

Example Images

[31 downloadable sample images and stacks](#) are available in ImageJ's *File>Open Samples* submenu. These images, and more, are also available as a [8.2MB zip archive](#).

<https://imagej.nih.gov/ij/download.html>

Feature Extraction

In this activity, we analyze an image of sand particles shown in Fig 1. That is, we determine each particle's

1. **Area**
2. Shape Descriptors namely **aspect ratio** (elliptic behavior), **roundness**, **circularity** and **solidity**
3. **Centroid locations** (x and y)

Note that each of these parameters and how they are calculated can be easily seen in the link: <https://imagej.nih.gov/ij/docs/menus/analyze.html>.

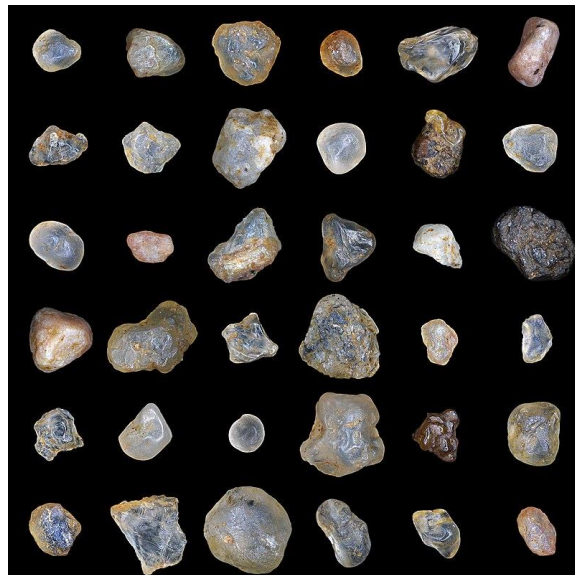


Fig 1. Image of sand particles [2]

In ImageJ, one can isolate the pixels of an image depending on their position with respect to the background (actually it's the pixel intensity, see the note in next slide). One can construct a histogram of such positions then introduce a threshold to cut off an image's background as shown in Fig 2. Of course, we convert the image to grayscale first since the software can only threshold one channel at a time!

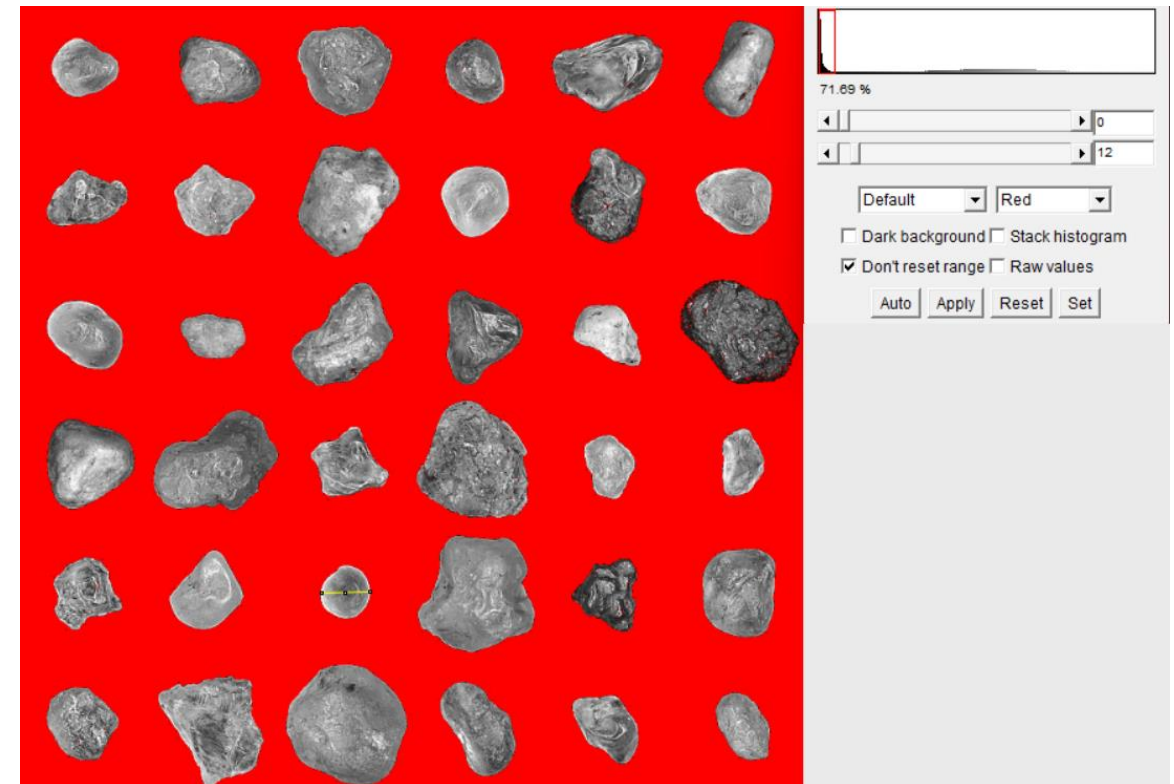


Fig 2. Removing the background by introducing threshold

Feature Extraction

Upon thresholding, we binarize the result (also in ImageJ) to get the following results Fig 3. Now, we can use the software to

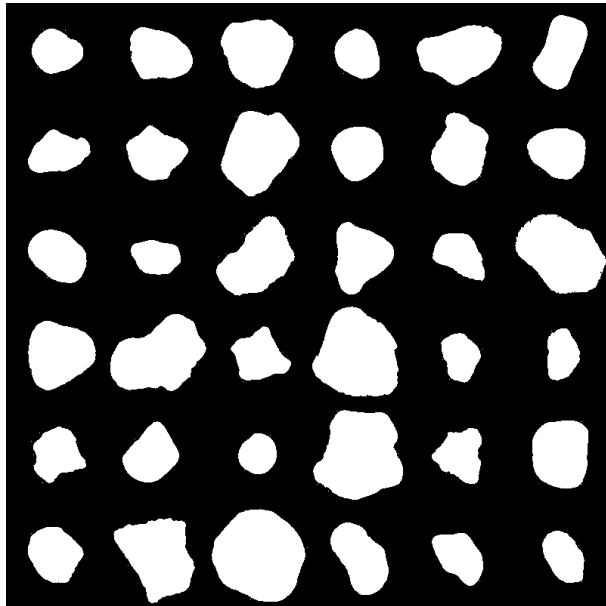


Fig 3. Binarized image of isolated sand particles

NOTE: Thresholding works by constructing a histogram of the intensities of each pixel. We then introduce a cutoff (sliders in ImageJ) to set the pixels below the threshold to a 0 value.

We can now use this image to analyze the objects using ImageJ's automated algorithms. We can test if the software successfully detected all blobs by checking if its count is correct. Since the thresholding is solely dependent on the user's judgement, one can confirm that the resultant shapes are satisfactory by isolating the outlines of each blob (also an option in ImageJ).

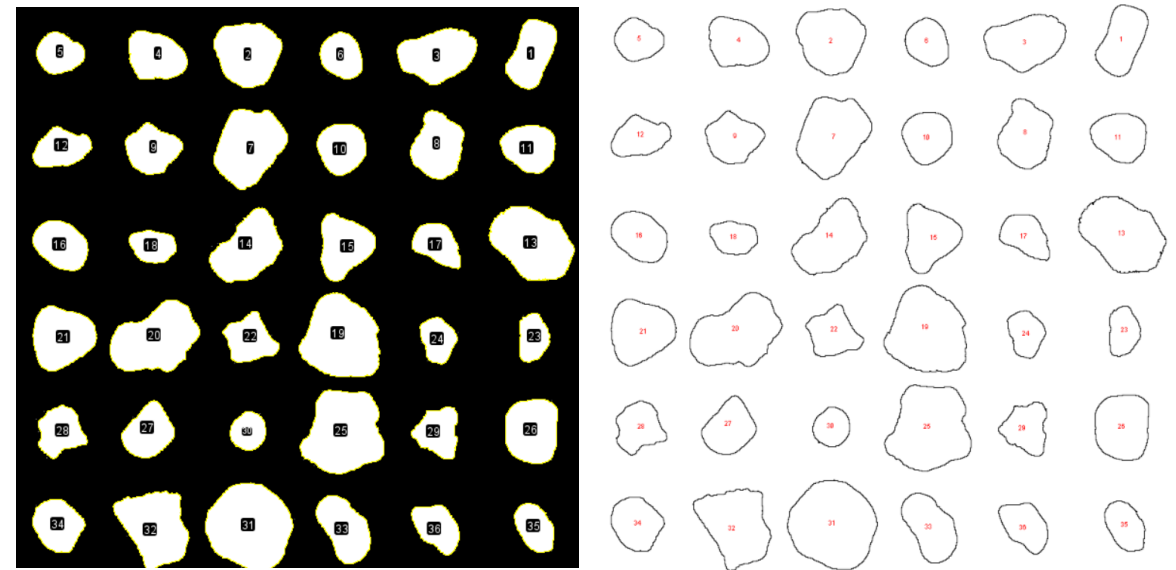


Fig 4. Counted particles (left) along with their outlines (right)

We can see that the software counted all blobs successfully and that the outlines more or less resemble the shapes in the original image.

Feature Extraction

Now we can use ImageJ's Analyze Particles option to extract the properties of each object. The data for the first 19 blobs are shown in Fig 5.

	Label	Area	StdDev	X	Y	Circ.	AR	Round	Solidity
1	sand.jpg	0.021	0	1.561	0.138	0.722	1.878	0.532	0.942
2	sand.jpg	0.03	0	0.707	0.137	0.853	1.048	0.954	0.97
3	sand.jpg	0.028	0	1.278	0.142	0.757	1.594	0.627	0.951
4	sand.jpg	0.02	0	0.431	0.145	0.81	1.343	0.745	0.969
5	sand.jpg	0.013	0	0.141	0.137	0.866	1.19	0.84	0.964
6	sand.jpg	0.014	0	0.99	0.145	0.872	1.201	0.832	0.972
7	sand.jpg	0.036	0	0.711	0.416	0.786	1.301	0.769	0.963
8	sand.jpg	0.023	0	1.278	0.416	0.759	1.324	0.755	0.94
9	sand.jpg	0.018	0	0.42	0.425	0.806	1.147	0.872	0.941
10	sand.jpg	0.017	0	0.99	0.421	0.894	1.057	0.946	0.977
11	sand.jpg	0.018	0	1.554	0.421	0.871	1.178	0.849	0.971
12	sand.jpg	0.015	0	0.146	0.423	0.75	1.584	0.631	0.935
13	sand.jpg	0.04	0	1.562	0.707	0.711	1.439	0.695	0.957
14	sand.jpg	0.03	0	0.707	0.723	0.725	1.583	0.632	0.931
15	sand.jpg	0.023	0	0.996	0.715	0.738	1.18	0.847	0.943
16	sand.jpg	0.018	0	0.146	0.713	0.873	1.368	0.731	0.973
17	sand.jpg	0.013	0	1.277	0.711	0.745	1.549	0.646	0.948
18	sand.jpg	0.011	0	0.422	0.716	0.836	1.466	0.682	0.963
19	sand.jpg	0.045	0	0.989	0.994	0.782	1.134	0.882	0.963

Fig 5. Data for each sand particle

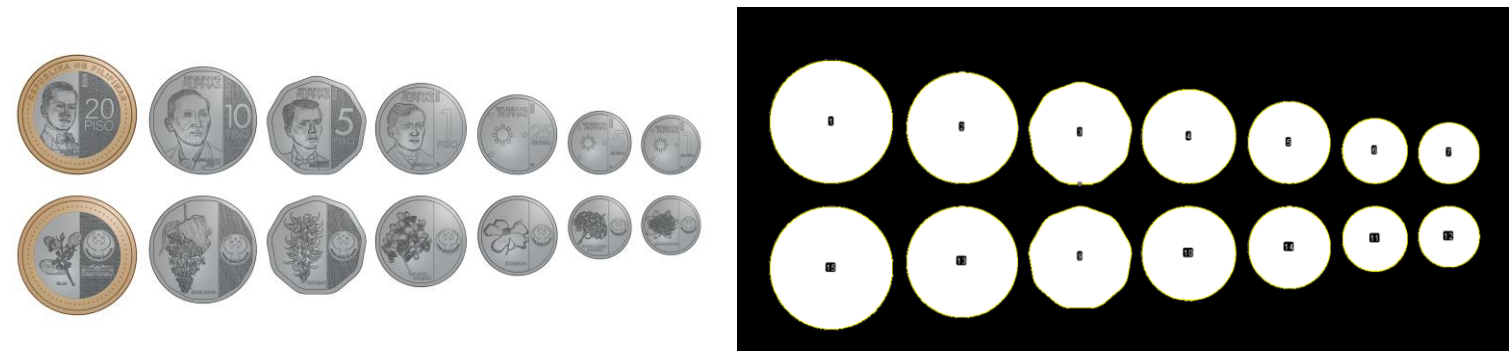
The method of computation for each quantity is available on the ImageJ website. I will display below the interpretation of each column instead.

1. Area – percentage with respect to the whole image
2. StdDev – standard deviation of pixel values (0 since we binarized)
3. X, Y – position of centroid pixel
4. Circ. – circularity, becomes more elongated as it goes to 0
5. AR – aspect ration, quotient of major and minor axis
6. Round – roundness, inverse of aspect ratio
7. Solidity – quotient of area and convex area

Now, let's apply the same technique to another image!

Feature Extraction

Here, we apply the same methods to describe the new coins release by the Bangko Sentral ng Pilipinas (BSP) [3]:



As seen in Fig 6., the results for circularity and aspect ratio reflects the fact that the coins are almost perfectly circular. The solidity is also close to 1, just as it should be!

	Label	Area	X	Y	Circ.	AR	Round	Solidity
1	coin.png	673.383	23.669	28.727	0.891	1.001	0.999	0.996
2	coin.png	550.146	54.996	30.181	0.895	1.001	0.999	0.996
3	coin.png	460.752	83.229	31.569	0.887	1.001	0.999	0.995
4	coin.png	394.594	109.206	32.226	0.818	1.001	0.999	0.994
5	coin.png	300.360	133.182	33.688	0.902	1.001	0.999	0.995
6	coin.png	188.141	153.684	35.687	0.676	1.002	0.998	0.987
7	coin.png	165.690	171.395	36.224	0.904	1.002	0.998	0.993
8	coin.png	0.320	83.251	43.604	0.028	112.000	0.009	1.000
9	coin.png	460.538	83.218	61.301	0.885	1.001	0.999	0.995
10	coin.png	394.631	109.196	60.216	0.819	1.001	0.999	0.994
11	coin.png	188.147	153.707	56.737	0.684	1.002	0.998	0.987
12	coin.png	165.767	171.424	56.222	0.899	1.001	0.999	0.993
13	coin.png	550.234	54.973	62.263	0.892	1.001	0.999	0.996
14	coin.png	300.403	133.266	58.809	0.897	1.001	0.999	0.995
15	coin.png	673.412	23.680	63.696	0.893	1.001	0.999	0.996

Now let’s see if the calculated area by ImageJ is accurate. The BSP lists the radius of their 20 pesos coin as 15 mm. Taking the area this translates to 706.86 mm^3.

This corresponds to object 1 and 15 in Fig 6. **The calculated error comes up to 4.67%.** Nice!

Maybe if I had a better threshold it would be more accurate. But frankly, my interest in these coins is now beginning to fade lol.

I also realized that we can apply this technology to making accurate masks in Fourier space! It truly is a useful tool.

Fig 6. Isolated and analyzed coins from the new series by the BSP

Key Takeaways

- ImageJ is a powerful tool for segmenting images and extracting features by the method of thresholding.
- The properties on an object can be estimated using an image and the appropriate scale bars by segmenting it.
- The accuracy of results outputted by ImageJ depends on the quality of thresholding done by the user.

Reflection

This activity was rather short, but is still very insightful and is a good close to the image processing chapter of our course. I learned what ImageJ is capable of and why it is a rather powerful tool, even though by today's best practices it isn't really cutting edge. This gave me a much needed perspective on the current state of image segmentation, and how powerful our tools have become like the recent release of the Segment Anything Model (SAM). The rapidly evolving field of computer vision is quite related to the laboratory I joined, and this activity is the first time that I was really able to dabble with it (sort of). I look forward to the next activities where we automate this task with machine learning! However even with its age, ImageJ is still able to calculate object parameters that are very useful. I wish I knew about this software in highschool because it would have made my research topic a million times better!

I was able to execute all procedures, do the extra challenge, and calculate parameters with errors less than 5%! Hence, I think I deserve a 100/100...

References

[1] Soriano, M. (2023). Feature Extraction from Images. App Physics 157. National Institute of Physics, UP- Diliman.

[2] Sand Particles Image: <https://en.wikipedia.org/wiki/Sand#/media/File:%D0%9F%D0%B5%D1%81%D1%87%D0%B8%D0%BD%D0%BA%D0%B8-1.%D0%96%D0%B5%D0%BB%D1%82%D1%8B%D0%B9%D1%81%D1%82%D1%80%D0%BE%D0%B8%D1%82%D0%B5%D0%BB%D1%8C%D0%BD%D1%8B%D0%B9%D0%BF%D0%B5%D1%81%D0%BE%D0%BA.jpg>

[3] Coins Image: <https://www.bsp.gov.ph/Pages/CoinsAndNotes/Coins/NewGenerationCurrencyCoinSeries.aspx>